

Erlang Overview

Consideriamo due paradigmi: **imperativo** e **funzionale**.

Funzionale → nessuna mutazione di stato, la computazione è valutare espressioni.

Imperativa → sequenza di comandi che operano sulle informazioni di stato. L'effetto dell'esecuzione di un comando (*istruzione*) è una modifica allo stato del programma.

Mancanza di uno stato → Le operazioni non dipendono da uno stato mutabile o condiviso.

Espressioni e sottoespressioni

Un calcolo è una sequenza di espressioni risultanti dalla valutazione (e *successiva sostituzione*) di sottoespressioni.

Lo stesso esempio di prima in programmazione funzionale è il seguente:

```
(15/3 if x > 0 else 2*3) - 1
=> (5 if 7 > 0 else 6 ) - 1
=> 5 - 1
=> 4
```

Valutazione + sostituzione → Per eseguire il programma, andiamo avanti step dopo step valutando ogni sottoespressione e sostituendo ogni sottoespressione con il suo valore corrispondente, alla fine otteniamo il risultato.

Referential transparency

Una funzione è detta **referenzialmente trasparente** se rimpiazzarla col suo valore finale non provoca effetti collaterali nel comportamento del programma.

Funzione pura → Le chiamate sono tutte referenzialmente trasparenti, quindi non hanno effetti collaterali.

Ovvero nell'esecuzione della funzione non vengono toccate altre cose esterne.

Memoization

Funzione pura → possiamo fare **memoization**.

Memorizziamo il risultato di una certa funzione (con un certo input) e le prossime volte che la troviamo mettiamo direttamente il valore senza ricalcolare.

Lack of state

Una variabile nel linguaggio funzionale è immutabile: una volta che assume un valore, non cambierà mai.

Possiamo creare nuove variabili, ma **non** modificare quelle già esistenti.

Infatti serve un **garbage collector** che si occupi di gestire oggetti non utilizzati per liberare la memoria.

Questa proprietà semplifica molto la gestione della concorrenza, dato che lo stato non può cambiare, non possono esserci data races:

Nessuno stato mutabile condiviso → Nessun problema nell'accesso simultaneo → Calcoli thread-safe (*nessuna critical race*)

First-class e Higher-order Functions

High-Order Functions → Possono accettare funzioni come argomenti e possono restituire funzioni come risultato.

First-Class Function → Funzione restituita come risultato da un'altra funzione.

Ricorsione

La ricorsione è a volte l'unico modo per portare avanti certi tipi di computazioni. Ciò che può essere fatto con l'iterazione può essere fatto anche con la ricorsione e viceversa.

Dato il lack of state, l'unico modo per ripetere gli step richiesti è utilizzare la ricorsione.

Nell'esempio sotto è il caso di una tail recursion, ovvero la ricorsione è l'ultima operazione.

```
def it_fact(n):  
    res = 1 #base case  
    while n>1:  
        res *= n  
        n -= 1  
    return res
```



```
def rc_fact(n, res=1):  
    #res acts as an "accumulator"  
    if n <= 1:  
        return res #base case  
    return rc_fact(n-1, res*n)
```

Tail optimization

Ogni volta che l'ultima operazione eseguita all'interno di una chiamata di funzione è un'altra chiamata di funzione, possiamo sostituire il frame del chiamante con il frame del chiamato, senza bisogno di mantenere entrambi.

Interprete ERTS

Erlang è eseguito da un interprete **ERTS**, come accade in Python e Java, utilizza un intermediario (*quindi è necessaria la compilazione*).

ERTS ha delle strutture di **garbage collection** per gestire la memoria dinamica.

BEAM

Il bytecode (*prende questo nome perchè le istruzioni sono codificate in un byte*) viene eseguito in una virtual machine detta BEAM.

Il BEAM è la macchina virtuale su cui viene eseguito il codice Erlang.

È il cuore del runtime di Erlang e si occupa di eseguire il codice compilato in un formato intermedio chiamato bytecode BEAM, garantendo efficienza, gestione della concorrenza, tolleranza ai guasti e altre caratteristiche chiave del linguaggio.

1. Codice Erlang → **bytecode BEAM (.beam)**.
2. bytecode caricato nella BEAM.

ERTS rende un programma Erlang **largamente indipendente dall'hardware sottostante**, grazie al fatto che il codice Erlang viene eseguito all'interno della macchina virtuale **BEAM**, che è parte integrante di ERTS.

Programma Erlang

Termine

L'espressione più semplice in Erlang è un **termine**.

La valutazione di un termine, restituisce il termine stesso. Un **termine** in Erlang è **qualsiasi valore valido nel linguaggio**.

Variabile

Una **variabile** è un'espressione.

Se una variabile è vincolata a un valore, la valutazione della variabile è il valore stesso.

Le variabili iniziano con una lettera maiuscola oppure con un underscore.

Le variabili sono **vincolate** ai valori tramite pattern matching (Tentativo di rendere identici un pattern (*lato destro*) e un termine (*lato sinistro*))

Pattern

Un pattern è strutturato come un termine, ma include variabili illimitate.

Atomo

Rappresentano valori costanti, in modo che il valore di un atomo sia univoco all'interno del programma (iniziano con lettere minuscole).

```
1> lion.  
lion  
2> Animal = lion.  
lion  
3> Animal = tiger.  
** exception error: no match of  
right hand side value tiger  
4> {zoo, Animal1, Animal2} = {zoo,  
'zebra joe', 'monkey bob'}.  
{zoo, 'zebra joe', 'monkey bob'}  
5> Animal1.  
'zebra joe'  
6> Animal2.  
'monkey bob'
```

Nella riga 2 in figura sotto vediamo che il valore a sinistra inizia con una lettera maiuscola, quindi è una variabile, mentre quello a destra è un atomo perchè inizia con la lettera minuscola.

Booleani non presenti → sono atomi true o false.

Tupla

Strutture con un **numero fisso di termini** e delimitate dalle {}.

Vengono utilizzate per raggruppare elementi, senza campi denominati.

I valori possono essere estratti dalle tuple utilizzando il pattern matching.

```
1> {alan,turing}.
{alan,turing}
2> Logician = {alonso, church,
{birthyear, 1903}}.
{alonso,church,{birthyear,1903}}
3> {Name,church,{birthyear,1903}}
= Logician.
{alonso,church,{birthyear,1903}}
4> Name.
alonso
5> {_, Lastname, _} = Logician.
{alonso,church,{birthyear,1903}}
6> Lastname.
church
```

Nella riga 1 abbiamo una tupla composta da due atomi.

Nella riga 3 abbiamo una tupla composta da una variabile che matchiamo con una variabile (Logician), un atomo e un'altra tupla.

Nella riga 5 vogliamo solo che la variabile Lastname prenda il valore di Logician.

Record

Variante "*denominata*" delle tuple, ovvero in cui i campi hanno un nome.

```
-record(game, {game_id, player1 , player2, ,game_over}).
```

Lista

Struttura con un **numero variabile di termini** (anche eterogenei).

$[E_1, \dots, E_N]$ è equivalente a $[E_1 \mid [\dots \mid [E_N \mid []]]]$

I valori possono essere estratti dagli elenchi utilizzando il pattern matching, anche nella forma $[H \mid T]$, dove H varrà come il primo elemento della lista e T come tutto il resto della lista.

```
1> L1 = [bob, 42, {4,2}].
[bob,42,{4,2}]
2> L2 = [3 | L1].
[3,bob,42,{4,2}]
3> [X|Y] = L1.
[bob,42,{4,2}]
4> X.
bob
5> Y.
[42,{4,2}]
6> [X,Z,W] = L1.
[bob,42,{4,2}]
7> W.
{4,2}
8> length(L1).
3
```

Riga 3 → X prenderà l'atomo (bob), mentre Y prenderà il resto della lista.

Riga 6 → Z prende il numero, mentre W prende la tupla.

Stringa

In Erlang le stringhe sono Liste di Interi, ogni intero allo UNICODE del carattere. C'è una sintassi speciale (*vengono utilizzati i delimitatori " "*):

```
1> S1 = "Hello".
"Hello"
2> S2 = S1 ++ " World!". %str-list concat
"Hello World!"
3> S2 ++ [10]. %10: newline
"Hello World!\n"
4> S2 ++ [-3].
[72,101,108,108,111,32,87,111,114,108,100,33,-3]
5> S3 = "\x{2200} e \x{2208} A \x{2a01} B".
[8704,32,101,32,8712,32,65,32,10753,32,66]
6> io:format("~ts~n",[S3]).
∀ e ∈ A ⊕ B
ok
```

Nella quarta linea, concatenare `s2` con un array composto da un solo numero che non ha una codifica di un carattere corrispondente, ovvero `-3`. A questo punto, `s2` viene vista come una lista di interi e `"-3"` viene regolarmente concatenato in fondo alla lista.

Utilizzando il formato mostrato nel comando sotto:

```
io:format("~ts~n" , [S3]) (~ts~n è il placeholder)
```

Possiamo visualizzare la stringa in un determinato template, nell'esempio, viene mostrato il simbolo corrispondente al codice assegnato ad `S3`.

List comprehension

Costrutto che consente di creare liste, filtrando ed elaborando elementi da una o più sorgenti (*liste o insiemi*).

Tecnica per manipolare collezioni di dati, applicando funzioni a **ogni** elemento della lista e includendo o escludendo elementi in base a condizioni.

```
[Espressione || Pattern <- Lista, Condizione]
```

- **Espressione:** valore che sarà inserito nella nuova lista.
- **Pattern <- Lista:** l'iterazione su ogni elemento della lista originale.
- **Condizione:** (opzionale) condizione che **filtra gli elementi della lista**. Solo gli elementi per cui la condizione è vera verranno inclusi nella nuova lista.

Creare una lista con i quadrati dei numeri da 1 a 5.

```
[X * X || X <- [1, 2, 3, 4, 5]]. % Risultato: [1, 4, 9, 16, 25]
```

Filtrare solo i numeri pari e calcolare i quadrati.

```
[X * X || X <- [1, 2, 3, 4, 5], X rem 2 == 0]. % Risultato: [4, 16]
```

BIF - Built-In Function

Funzioni "integrate" all'interno di BEAM.

Ad esempio `date()` restituisce la data, nella riga 4 vediamo una built-in function `element()` che consente di ottenere un elemento di una tupla. Nella riga 7 la funzione `io:format()` consente di stampare a video.

```

1> date() .
{2022,10,10}
2> time() .
{10,11,42}
3> Tup = {lion,rhino,ostrich} .
{lion,rhino,ostrich}
4> element(2,Tup) .
rhino
5> Lis = tuple_to_list(Tup) .
[lion,rhino,ostrich]
6> length(Lis) .
3
7> io:format("hello~n") .
hello
ok

```

Funzioni

All'interno di un modulo è possibile definire delle funzioni (*pubbliche o private*).

Le funzioni pubbliche possono essere richiamate da altri moduli, mentre quelle private sono accessibili esclusivamente all'interno del modulo stesso.

Una funzione pubblica deve essere dichiarata esplicitamente nell'attributo `-export`, che specifica l'elenco delle funzioni visibili all'esterno del modulo.

In Erlang, una funzione è definita come una sequenza di clausole, separate da punti e virgola (;) e terminate da un punto (.).

Ogni clausola specifica un comportamento in base ai pattern dei suoi argomenti e, opzionalmente, può includere una guardia introdotta dalla keyword `when`. Il corpo di una clausola è una sequenza di espressioni, separate da (,).

Una **clausola** corrisponde a ciascun ramo della definizione della funzione, identificato da un pattern (*gli argomenti tra parentesi*) e dal relativo corpo.

```
fact(0) -> 1;
```

```
fact(N) when N > 0 -> N * fact(N - 1).
```

Queste qua sopra sono due clausole della funzione `fact`.

Quando il pattern corrisponde, il relativo corpo della clausola viene eseguito e il risultato viene restituito come risultato della funzione.

Il nome di una funzione è un atomo. Gli argomenti di una funzione sono espressi mediante **pattern matching**, che consente di destrutturare i dati in ingresso.

All'interno di un modulo, una funzione è identificata dalla coppia nome/arity.


```
-module(factorial).
-export([fact/1]).
% Caso base: il fattoriale di 0 è 1
fact(0) -> 1;
% Caso ricorsivo: N! = N * (N-1)!
fact(N) when N > 0 -> N * fact(N - 1).
```

c(factorial).

factorial:fact(5). % Output: 120

Controllo del flusso - Guardia

Espressione booleana utilizzata per controllare il flusso di esecuzione, aumentando la potenza del **pattern matching**.

Formalmente, una guardia è una sequenza di **espressioni** progettate per essere sicure e prive di effetti collaterali: pertanto, non è consentito utilizzare funzioni definite dall'utente all'interno delle guardie.

Nell'esempio specifichiamo una funzione per ottenere il segno di un certo numero.

```
sign(X) when is_number(X), X>0 ->
1;
sign(X) when is_number(X), X==0 ->
0;
sign(X) when is_number(X) ->
-1;
sign(_) -> argerror.
```

Controllo del tipo di dato: Le variabili in Erlang non sono tipizzate quindi `x` può essere qualsiasi cosa, per controllare se una variabile è un numero utilizzeremo `is_number(x)`.

In questo caso, la guardia è la sequenza di espressioni "`when is_number(x)`".

1. **Controllo del segno:** Se `x > 0` → restituisce 1.
2. **Controllo se 0:** Se `x = 0` → restituisce 0.
3. **Controllo del segno:** Se `x < 0` → restituisce -1.

In tutti gli altri casi (se `x non numero`) restituisce un errore (*atomo*). Viene utilizzata la **variabile senza nome** "`_`", per valutare il segno in questo caso.

Espressione Case

```
case Expr of
  P1 [when C1] -> E1;
  :
  Pn [when Cn] -> En
end
```

L'espressione **Expr** viene valutata in **T** e i pattern **P1 ... Pn** vengono confrontati in sequenza con **T**.

Non appena si verifica una corrispondenza e la relativa guardia (*facoltativa*) è vera, viene valutata la corrispondente **Ei** come valore di ritorno del **case**.

Se però non c'è un pattern **catch-all** alla fine, viene restituito un errore in caso nessuno dei precedenti pattern sia rispettato (*corrisponde al default nello switch-case*).

```
-module(case_example).
-export([check_number/1]).

check_number(N) ->
  case N of
    0 -> zero;
    1 -> one;
    _ when N > 1 -> positive;
    _ -> negative
  end.
```

Espressione If

Un **if** è una **sequenza di guardie** valutate una dopo l'altra.

Valutate in sequenza → Non appena una guardia **Guard_i** “riesce”, il valore di ritorno è il valore della relativa clausola **Ei**, quelle successive **non** vengono valutate.

```
if
  Guard1 -> E1;
  :
  Guardn -> En
end
```

Se nessuna guardia riesce viene lanciato errore di runtime `if_clause`, per evitare questo errore, spesso viene inserita una guardia finale (*clausola di default*) sempre `true`.

```
test(X) ->
  if
    X > 10 ->
      io:format("X è maggiore di 10~n"),
      greater_than_10;
    X > 5 ->
      io:format("X è maggiore di 5~n"),
      greater_than_5;
    true -> % Clausola di default
      io:format("X è 5 o meno~n"),
      less_than_or_equal_5
  end.
```

Creazione di un Loop

Non esistono istruzioni `loop` o `for` o `while` native in erlang, ma implementarle da 0 è facile.

```
-module(helloworld) .
-export([forhw/1,start/0]) .

forhw(0) -> done;
forhw(N) ->
  io:fwrite("Hello~n"),
  forhw(N-1) .

start() ->
  forhw(5) .
```

`forhw(N)` serve per specificare quante volte (`N`) dobbiamo ripetere una certa azione.

È una funzione ricorsiva: fa qualcosa e richiama se stessa per altre `N-1` volte. Nel caso `forhw(0)` non vengono fatte più iterazioni.

Loop per scorrere una lista

```
-module(showlist).  
-export([scanl/1,start/0]).  
  
scanl(L) -> scanl(L,0).  
  
scanl([], Index) -> Index;  
scanl([H|T], Index) ->  
    io:fwrite("~w: ~w~n",[Index,H]),  
    scanl(T,Index+1).  
  
start() ->  
    X = [10,2,7,4],  
    scanl(X).
```

Quando si scrivono funzioni ricorsive, si parte sempre dal caso base.

Caso base → Lista vuota → viene restituito **Index**.

Caso normale → dividiamo in testa (H) e coda (T) → ricorsione su T incrementando **Index**.

Quando la coda diventa vuota → caso base.

Espressione fun

TempConv = fun(...)

fun è un modo per rappresentare una funzione tramite una variabile.

```
1> TempConv = fun({cel,C}) -> {far, 32 + C*9/5};  
1>           ({far,F}) -> {cel, (F-32)*5/9}  
1>           end.  
#Fun<erl_eval.6.128620087>  
2> TempConv({cel,22}).  
{far,71.6}  
3> TempConv({far,0}).  
{cel,-17.77777777777778}
```

High order functions - Funzione come parametro di ingresso

Le **funzioni di alto livello** in Erlang consentono di esprimere operazioni generiche con un livello di astrazione di controllo, delegando dettagli specifici a una funzione passata come parametro.

La funzione di alto livello è **myfilter/2**.

```
myfilter( _ , [] ) -> [];
myfilter(Pred,[H|T]) -> case Pred(H) of
  true -> [H|myfilter(Pred,T)];
  false -> myfilter(Pred,T)
end.
```

`Pred` è un **predicato**, ovvero una funzione che deve restituire gli atomi `true` o `false`.

Applichiamo il predicato alla testa `H`.

- `true` → testa mantenuta.
- `false` → richiamato il filtro sulla coda, scartando la testa.

Vediamo il nostro predicato, ovvero `is_even/1` la quale controlla se `x` è dispari.

```
is_even(X)-> case (X rem 2) of
  0 -> true;
  _ -> false
end.
```

Per passare il predicato alla funzione di alto livello basta specificare che il parametro è una funzione con `fun`:

```
test() -> myfilter( fun is_even/1 ,
[1,2,3,4,5] ).
```

Quindi la funzione "`is_even/1`" viene passata come parametro di `my_filter` per implementare il predicato `Pred`.

High order functions - Funzione come valore di ritorno

Siano due funzioni come parametri, `F()` e `G()`.

Come risultato viene restituita un'altra funzione che è la composizione delle due (`F(G(X))`).

Functions can be returned by functions as well.

In the example, `fcomp` returns the composition of two functions (with arity 1) passed as input.

```
-module(mymod).
...
fcomp(F,G) ->
  fun(X) -> F(G(X)) end.
...
```

shell

```
1> Myf1 = mymod:fcomp(fun(X)->X*X end,
fun(X)->X+1 end).
#Fun<mymod.1.70048917>
2> Myf2 = mymod:fcomp(fun(X)->X+1 end,
fun(X)->X*X end).
#Fun<mymod.1.70048917>
3> io:format("~p\n",[Myf1(5)]).
36
ok
4> io:format("~p\n",[Myf2(5)]).
26
ok
```

Function values
as params

Che cos'è OTP?

OTP - Open Telecom Platform funziona da middleware per le applicazioni Erlang.

Si tratta di un insieme di librerie e strumenti che fornisce un framework per semplificare la costruzione di applicazioni **concorrenti, distribuite e fault-tolerant**.

Fornisce:

- **Gestione dei processi:** con `gen_server`, `supervisor`, e altri behaviour.
- **Supervisione e tolleranza agli errori:** grazie a una gerarchia di supervisor che riavviano i processi in caso di crash.
- **Gestione del codice e degli aggiornamenti a caldo** (*hot code swapping*).
- **Strumenti per la distribuzione di applicazioni** su più nodi.

Un concetto fondamentale nello sviluppo di applicazioni OTP è il **behavior**.

Behaviour

Un **behaviour** in Erlang definisce un **insieme di funzioni (callback) che un modulo deve implementare**. Similmente a un'interfaccia in Java.

A differenza delle interfacce, i behavior devono fornire una implementazione generica, non solo la definizione.

Un behavior è più simile ad una classe astratta, quindi fornisce un comportamento base.

Il comportamento `supervisor` gestisce la supervisione dei processi, in modo da garantire che i processi vengano riavviati automaticamente in caso di errore.

Riusabilità: I comportamenti forniscono moduli generici che possono essere facilmente riutilizzati.

Che cos'è `gen_server`?

`gen_server` è un **comportamento OTP** (un "behavior"), un modulo predefinito che implementa un server generico.

Ti permette di creare facilmente **processi concorrenti** che gestiscono richieste sincrone e asincrone senza dover implementare manualmente tutta la gestione dei messaggi.

Per cosa si usa?

Creare **server di lunga durata** che gestiscono richieste e mantengono uno stato.

In pratica, `gen_server` incapsula un loop che:

1. Riceve messaggi dai client.
2. Processa le richieste (sincrone o asincrone).
3. Mantiene uno **stato interno**.
4. Risponde ai client, se necessario.

Un `gen_server` **deve** implementare alcune funzioni specifiche. Queste funzioni sono i **callback obbligatori** che `gen_server` si aspetta di trovare nel modulo. In caso contrario, il codice non funzionerà correttamente.

- **`init/1`**
 - Inizializza lo stato del server.
 - Deve restituire `{ok, State}` oppure `{ok, State, Timeout}`.
- **`handle_call/3`**
 - Gestisce le richieste **sincrone** (`gen_server:call/2`).
 - Deve restituire `{reply, Risposta, NuovoStato}` o altre varianti.
- **`handle_cast/2`**
 - Gestisce le richieste **asincrone** (`gen_server:cast/2`).
 - Deve restituire `{noreply, NuovoStato}` o altre varianti.
- **`handle_info/2`**
 - Gestisce messaggi generici che non sono né `call` né `cast`.
 - Deve restituire `{noreply, NuovoStato}` o altre varianti.
- **`terminate/2`**
 - Viene chiamata quando il server si sta terminando.
 - Serve per pulire risorse, chiudere file, ecc.

Cosa sono le callbacks?

Definizione Generale:

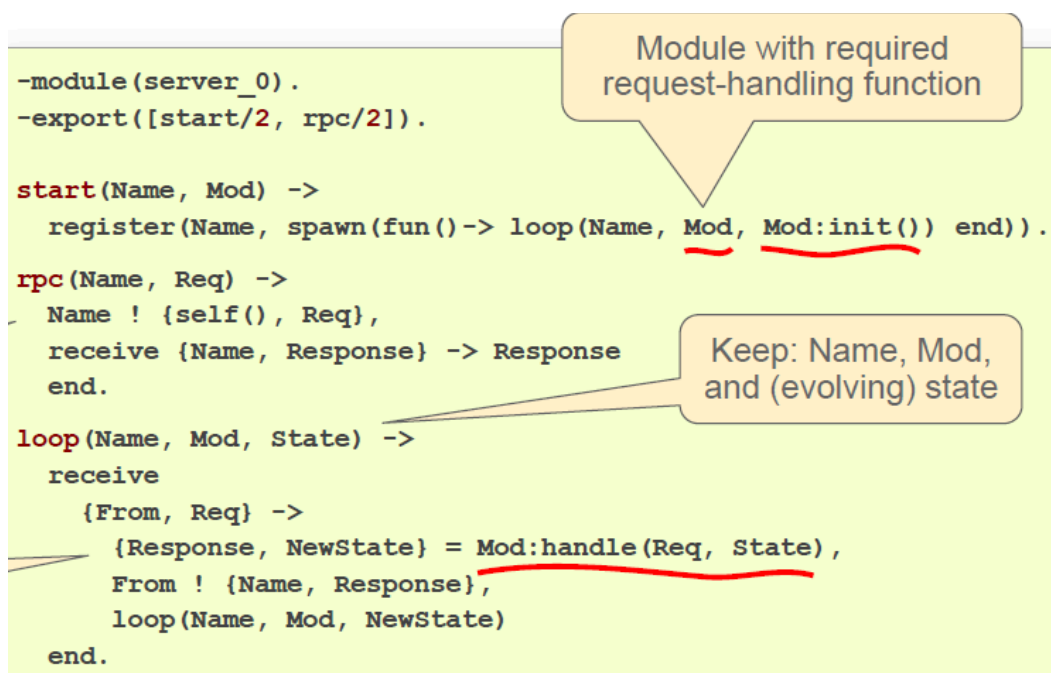
Funzione che viene **passata come argomento** a un'altra funzione e chiamata in un secondo momento, solitamente in risposta a un evento o quando un'operazione è completata.

Un esempio di funzione di callback è la funzione `onclick` in JavaScript, viene chiamata solo in caso dell'evento click del mouse.

Callback nel contesto dei `gen_server`

Sono quelle che il comportamento OTP chiama automaticamente per gestire richieste e eventi.

Il programmatore non chiama direttamente queste funzioni, ma deve implementarle (*invece che fornire come argomento, ne implementa il codice*) in un modulo affinché `gen_server` possa eseguire il proprio ciclo di elaborazione.



In questo esempio il modulo `Mod` che passiamo nella funzione `loop` è un modulo personalizzato.

Dobbiamo definire il modulo personalizzato con le **funzioni di callback** `init()` e `handle(Req, State)`.

Importante: la funzione di callback effettiva deve essere identificata tramite argomenti effettivi e talvolta anche in base alle convenzioni di denominazione.

- **Callback sincroni:** Eseguiti prima che la funzione chiamata restituisca il risultato.

- **Callback differiti:** Eseguiti dopo; utilizzati per la gestione degli eventi, GUI, ecc.

Parametrizzare un server con una funzione di callback

Qui specifichiamo un server che riceve un numero per richiesta e restituisce la media di tutti i valori ricevuti finora.

```
-module(avg_server).  
-export([init/0, handle/2, get_avg/1]).  
-import(server_0, [rpc/2]).  
  
% client routines  
get_avg(X) -> rpc(calc_avg, X).  
.  
  
% callback routines  
init() -> {0, 0}. % total: 0; # of reqs: 0  
  
handle(X, {Total, Times}) ->  
    NewTotal = Total + X,  
    NewTimes = Times + 1,  
    {NewTotal/NewTimes, {NewTotal, NewTimes}}.
```

The diagram illustrates the code with three callout boxes:

- Server name to be registered:** Points to the `avg_server` module name in the `-module` line.
- Initial state:** Points to the `init()` function definition.
- State info: Total, and # of calls:** Points to the `handle` function definition.

Impostazione del server: chiamare `start/2` dal comportamento, fornendo:

- nome standard per il server.
- riferimento al modulo di callback.

Il server viene utilizzato tramite routine client, definite nel modulo di callback.

ESERCIZI ERLANG

implementa una funzione erlang che riceve una lista, decreta se questa è palindroma oppure no

```
-module(palindrome).
-export([is_palindrome/1]).

is_palindrome(List) ->
    List == reverse(List).

% Funzione per invertire una lista (senza lists:reverse/1)
reverse(List) -> reverse(List, []).

reverse([], Acc) -> Acc;
reverse([H | T], Acc) -> reverse(T, [H | Acc]).
```

Oppure:

```
-module(palindrome).
-export([is_palindrome/1]).
is_palindrome(List) ->
    List == reverse(List, length(List))
reverse(List, 0) -> List;
reverse([H | T], N) ->
    reverse([T | H], N-1).
```

Implementare una funzione di riduzione.

```
%%implementare una funzione di riduzione
-module(exercise7).
-export([reduce/3]).

% Caso base: se la lista è vuota, restituiamo l'accumulatore
reduce(_Fun, Acc, []) ->
    Acc;
% Caso ricorsivo: applichiamo la funzione e passiamo al resto della lista
reduce(Fun, Acc, [Head | Tail]) ->
    reduce(Fun, Fun(Head, Acc), Tail).

% Somma di una lista di numeri
Sum = my_reduce:reduce(fun(X, Acc) -> X + Acc end, 0, [1, 2, 3, 4, 5]).
% Risultato: 15

% Concatenazione di una lista di stringhe
Concat = my_reduce:reduce(fun(X, Acc) -> X ++ Acc end, "", ["Hello", " ", "World"]).
% Risultato: "Hello World"
```

Oppure:

```
-module(sum).  
-export([sum/1]).  
sum(List) -> sum(List, 0).  
sum([], Cont) -> Cont;  
sum([H | T], Cont) ->  
    sum(T, Cont + H).
```

How would you implement a quicksort in Erlang

```
%ordinare una lista (quicksort)  
-module(exercise1).  
-export([quicksort/1]).  
  
quicksort([]) -> [];  
quicksort([Pivot | Rest]) ->  
    Smaller = [X || X <- Rest, X <= Pivot],  
    Greater = [X || X <- Rest, X > Pivot],  
    quicksort(Smaller) ++ [Pivot] ++ quicksort(Greater).
```

```
-module(quicksort).  
-export([quicksort/1]).  
quicksort([]) -> [];  
quicksort([Pivot | Rest]) ->  
    Greater = [X || X <- Rest, X > Pivot],  
    Smaller = [X || X <- Rest, X <= Pivot],  
    quicksort(Smaller) ++ [Pivot] ++ quicksort(Greater).
```

1. scrivere una funzione invert che restituisce la lista invertita

```

-export([reverse/1]).

-export([reverse2/1]).

%%caso lista vuota
reverse([]) -> [];

reverse([H | T]) -> reverse(T) ++ [H].

%[1 2 3]

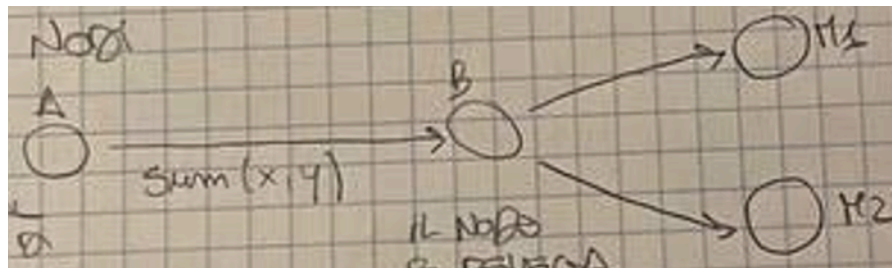
% reverse([3 | ]) -> reverse() ++ [3] ----> [3]
% reverse([2 | 3]) -> reverse(3) ++ [2] ---> [3 2]
% reverse([1 | 2 3]) -> reverse(2 3) ++ [1] ---> [3 2 1]

%%caso con accumulatore (tail recursion)
reverse2([], R) -> R;
reverse2(L, R) -> reverse2(L, [H|R]).

reverse2([], R) -> R;
reverse2([H | T], R) -> reverse2(T, [H|R]).

```

Due nodi:



A chiede al nodo B di fare una somma.

Il nodo B delega l'operazione al nodo M1 se x è pari mentre a M2 altrimenti.

Scrivere la struttura (codice) del nodo B: fare attenzione a chi deve rispondere al nodo A (B attenda la risposta del nodo M1 (M2) tramite la receive, quindi risponde al nodo A.

Risposta: La condizione si può fare con `x rem 2 == 0` (tramite delle guardie sul pattern matchino del messaggio).

```

nodeB() ->
  receive
    {X, Y, PidA} when X rem 2 == 0 ->
      PidM1 ! {X, Y, self(), PidA};

    {X, Y, PidA} when X rem 2 /= 0 ->
      PidM2 ! {X, Y, self(), PidA},

    {sum, Sum, PidA} -> PidA ! {sum, Sum}
  end.

nodeA(PidB, X, Y) ->
  pidB ! {X, Y, self()},
  receive
    {sum, Sum} -> io:format("The sum is "++Sum);
  end.

```

```

nodeM1() ->

  receive
    {X, Y, PidB, PidA} ->
      Sum = X + Y,
      PidB ! {sum, Sum, PidA}
  end.

nodeM2() ->

  receive
    {X, Y, PidB, PidA} ->
      Sum = X + Y,
      PidB ! {sum, Sum, PidA}
  end.

```

2. fare un semaforo

```

%%semaforo
-module(exercise5).
-export([start/1, acquire/1, release/1]).

%% Avvia il semaforo con un numero massimo di risorse disponibili
start(Max) ->
    spawn(fun() -> loop(Max) end).

%% Processo che gestisce lo stato del semaforo
loop(Count) ->
    receive
        {acquire, Pid} when Count > 0 ->
            Pid ! ok,
            loop(Count - 1);
        {acquire, Pid} ->
            receive
                {release} -> loop(Count + 1)
            after 0 ->
                Pid ! wait,
                loop(Count);
            end;
        {release} ->
            loop(Count + 1)
    end.

```

```

%% Funzione per acquisire il semaforo
acquire(Sem) ->
    Sem ! {acquire, self()},
    receive
        ok -> ok;
        wait -> acquire(Sem) % Riprova finché non riesce
    end.

%% Funzione per rilasciare il semaforo
release(Sem) ->
    Sem ! {release}.

```

```

-module(semaphore) .

```

```

-export([semaphore/1]).
start(N) ->
    semaphore(N).
semaphore(N) ->
    receive
        {acquire, Pid} when N > 0 ->
            Pid ! ok,
            semaphore(N-1);
        {acquire, Pid} ->
            Pid ! wait,
            semaphore(N);
        {release} ->
            semaphore(N+1);
        _ ->
            semaphore(N)
    end.

-module(proc).
-export([acquire/0, release/0]).
acquire() ->
    Pid = where_is(Server),
    Pid ! {acquire, self()},
    receive
        {ok} ->
            io:format("Lock acquired~n"),
            release();
        {wait} ->
            io:format("Wait~n"),
            acquire();
        _ ->
            io:format("Unknown Message~n")
    end.
release() ->
    Pid = where_is(Server),
    Pid ! {release}.

```

3. moltiplicare tutti i numeri di una lista

```

%% moltiplicare tutti gli elementi di una lista
-module(exercise2).

-export([rmul/1]).

rmul([]) -> 1;

%%in questo caso è ricorsiva senza utilizzare tail recursion
rmul([H|T]) -> H * rmul(T).

%%utilizzo di tail recursion utilizzando un accumulatore
rmul(L) -> rmul(L, 1).
rmul([H|T], Acc) -> rmul(T, Acc * H);
rmul([], Acc) -> Acc.

```

4. estrarre solo numeri pari o dispari da una lista

```

-export([filter_even/1]).
-export([filter_even/2]).

-export([filter_odd/1]).
-export([filter_odd/2]).

%%solo pari
filter_even([]) -> [];
filter_even([H | T]) when H rem 2 == 0 -> [H | filter_even(T)];
filter_even([_ | T]) -> filter_even(T).

%%con list comprehension
filter_even(List) ->
    [X || X <- List, X rem 2 == 0].

%%solo dispari
filter_odd([]) -> [];
filter_odd([H | T]) when H rem 2 != 0 -> [H | filter_odd(T)];
filter_odd([_ | T]) -> filter_odd(T).

%%con list comprehension
filter_odd(List) ->
    [X || X <- List, X rem 2 != 0].

```


Oppure:

```
-module(extract).
-export([extract/2]).
extract(List, Val) ->
    case Val of
        1 -> [ X || X <- List, X rem 2 != 0]; % 1 per dispari
        0 -> [ X || X <- List, X rem 2 == 0]; % 0 per pari
        _ -> [] %altri valori ritornano lista vuota
    end.
```

5. date due liste A e B, ritornare solo gli elementi di A che non compaiono in B

```
-module(exercise4).

-export([difference/2]).
%%soluzione con list:member
difference([], _) -> [];
difference([H | T], B) when lists:member(H, B) -> difference(T, B);
difference([H | T], B) -> [H | difference(T, B)].

%%soluzione senza list member
-module(list_utils).
-export([difference/2]).

difference([], _) -> [];
difference([H | T], B) when is_in_list(H, B) -> difference(T, B);
difference([H | T], B) -> [H | difference(T, B)].

is_in_list(_, []) -> false;
is_in_list(X, [X | _]) -> true;
is_in_list(X, [_ | T]) -> is_in_list(X, T).
```

```
-module(extract).
-export([extract/2]).
extract(List1, List2) ->
    [ X || X <- List1, is_in_list(X, List2)].

is_in_list(X, []) -> false;
is_in_list(X, [H | T]) ->
    if
        X == H -> true;
        true -> is_in_list(X, T)
    end.
```